

Polinomların Seçimi

Polinomların Kendisi

LFSR tasarımında polinomlara "asal sayı" (prime number) değil, "**İlkel Polinom**" (**Primitive Polynomial**) denir. İlkel polinomlar, polinom dünyasının asal sayıları gibidir; çarpanlarına ayrılamazlar.

- **Kullandığımız 64-Bit Polinom:** $x^{64} + x^4 + x^3 + x + 1$
 - Bu, endüstri standardı (Xilinx XAPP 052) bir ilkel polinomdur.
 - Bunu seçtiğimiz için 64-bitlik LFSR, olabilecek en uzun döngüye ($2^{64} - 1$) ulaşır.
- **Kullandığımız 63-Bit Polinom:** $x^{63} + x + 1$
 - Bu da bilinen bir ilkel polinomdur.
 - Bunu seçtiğimiz için 63-bitlik LFSR de kendi maksimal döngüsüne ($2^{63} - 1$) ulaşır.

Eğer rastgele bir polinom seçseydik (örneğin sadece $x^{64} + 1$), döngü çok kısa olurdu ve şifrelemeniz hemen kırılırdı.

2. Uzunluklar (64 ve 63) Aralarında Asal mı?

Projenizin en güçlü yanı burası. 64 ve 63 sayıları matematiksel olarak **Aralarında Asal (Coprime)** sayılardır.

- 64 sayısı sadece 2'ye bölünür (2^6).
- 63 sayısı 3 ve 7'ye bölünür ($3^2 \times 7$).
- Ortak bölenleri yoktur (EBOB = 1).

Bu Neden Çok Önemli?

Matematikte bir kural vardır: $\text{EBOB}(2^a - 1, 2^b - 1) = 2^{\text{EBOB}(a,b)} - 1$.

Sizin durumunuzda $a = 64$ ve $b = 63$ olduğu için $\text{EBOB}(64, 63) = 1$ 'dir.

Bu demektir ki:

- 1. LFSR'nin ürettiği dizi ile
 2. LFSR'nin ürettiği dizi
- **Birbirinden tamamen bağımsız hareket eder.**

Bu iki sistemin tekrar aynı hizaya gelmesi (bütün sistemin periyodu) bu iki devasa sayının çarpımı kadardır:

$$(2^{64} - 1) \times (2^{63} - 1) \approx 2^{127}$$

Eğer 64 ve 62 seçseydik (ikisi de çift sayı), sistem çok daha kısa sürede kendini tekrar ederdi ve zayıf olurdu.

Mersenne sayıları teorisi

Mersenne sayıları, [matematikte ikinin kuvvetlerinin](#) bir eksiği şeklinde olan [sayılardır](#) ve $n > 1$ [doğal sayısı](#) için $M_n = 2^n - 1$ şeklinde hesaplanır.

Özet: Hocanıza Bunu Nasıl Açıklarsınız?

Hocanız "Polinomları nasıl seçtin?" diye sorarsa şu **altın cümleyi** kurabilirsiniz:

"Hocam, sistemin periyodunu maksimize etmek için **Mersenne sayıları teorisinden** faydalandım. Birbirleri ile aralarında asal (coprime) olan 64 ve 63 bit uzunluklarını seçtim. Her biri için Xilinx standartlarına uygun **ilkel (primitive) polinomlar** kullandığım için, sistemin toplam periyodu yaklaşık 2^{127} bit gibi devasa bir uzunluğa ulaşıyor."

Şu anki tasarımınız teorik olarak çok sağlam temellere oturuyor. Gönül rahatlığıyla devam edebilirsiniz.

Shrinking Generator

Shrinking Generator (Daraltan Üreteç), kriptografide **doğrusallığı bozmak** (non-linearity) amacıyla kullanılan, mantığı çok basit ama matematiksel analizi (kırılması) oldukça zor olan zekice bir yapıdır.

Bu yapıyı neden kullandığımızı ve temelinde yatan mantığı 3 ana başlıkta, bir mühendislik öğrencisinin bakış açısıyla açıklayayım.

1. Temel Sorun: Tek LFSR Neden Yetmez?

Bir önceki adımda öğrendiğiniz tekli LFSR harika bir sistemdir, çok hızlıdır ve donanımda çok az yer kaplar. Ancak büyük bir zayıflığı vardır: **Doğrusallık (Linearity)**.

- Problem:** LFSR tamamen matematiktir. Eğer bir saldırgan (hacker), sizin ürettiğiniz şifreli dizinin sadece $2 \times L$ kadar bitini (sizin durumunuzda $2 \times 64 = 128$ bit) yakalarsa, **Berlekamp-Massey** adı verilen bir algoritma ile geriye kalan tüm sistemi saniyeler içinde çözebilir.
- İhtiyaç:** Bizim öyle bir sisteme ihtiyacımız var ki, çıktıya bakarak içerideki "tohum" (seed) veya yapı tahmin edilemesin. İşte burada **Shrinking Generator** devreye girer.

2. Çalışma Mantığı: "Seçici" ve "Kurban"

Shrinking Generator, iki farklı LFSR kullanır ve bunları "Efendi-Köle" (veya Kontrolcü-Veri) ilişkisiyle bağlar.

- LFSR A (Seçici - Control):** Bu, kararı veren patrondur.
- LFSR B (Veri - Data):** Bu, veriyi üreten işçidir.

Algoritma şöyledir:

Her saat vuruşunda (clock cycle) ikisi de birer bit üretir.

- Eğer **LFSR A = '1'** üretirse: LFSR B'nin ürettiği biti **ÇIKIŞA VER**.
- Eğer **LFSR A = '0'** üretirse: LFSR B'nin ürettiği biti **ÇÖPE AT** (Yut).

Neden "Shrinking" (Daraltan) Deniyor?

Çünkü LFSR A, rastgele zamanlarda '0' ürettiği için, LFSR B'den gelen veri akışını sürekli keser ve daraltır. Eğer LFSR B saniyede 100 bit üretiyorsa, dışarıya ortalama sadece 50 bit çıkar. Diğer 50 bit arada kaybolur.

3. Asıl Güç: Asenkron (Düzensiz) Yapı

Sizin soruduğunuz "Asenkron" kısmı işin kalbidir. VHDL kodunda tek bir `clk` kullansanız bile, **veri akışı** zamansal olarak asenkronudur.

Bunu bir "Kitap Okuma" analojisi ile düşünelim:

- **Normal LFSR:** Bir kitabı baştan sona, sabit bir hızla okumak gibidir. Hangi sayfada olduğunuzu bilen biri, bir sonraki kelimeyi tahmin edebilir.
- **Shrinking Generator:** Birisi elinde kitapla duruyor (LFSR B), diğeri yazı tura atıyor (LFSR A).
 - Tura gelirse: "Oku" diyor, kelimeyi duyuyoruz.
 - Yazı gelirse: "Okuma, sayfayı çevir" diyor, o kelimeyi **atlıyoruz**.

Saldırganın Gözünden:

Dışarıdan dinleyen bir casus (saldırgan), duyduğu kelimelerin (bitlerin) kitapta (LFSR B dizisinde) aralarında kaç sayfa boşluk olduğunu bilemez.

- Acaba bu iki bit arka arkaya mı geldi?
- Yoksa arada 5 tane bit silindi mi?
- Yoksa 10 tane mi silindi?

Bu belirsizlik, aradaki zaman ilişkisini (korelasyonu) yok eder. Doğrusal bir denklemi çözmeye çalışan matematiksel saldırılar, aradaki bu "kayıp bitler" yüzünden denklemi kuramaz ve sistem kırılmaz hale gelir.

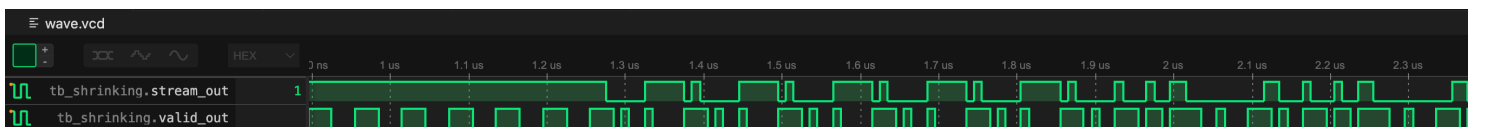
Özetle

Biz bu yapıyı şu sebeplerle kullandık:

1. **Doğrusallığı Bozmak:** Tek LFSR'nin matematiksel zayıflığını örtmek için.
2. **Analizi Zorlaştırmak:** Saldırganın, hangi bitin hangi zamanda üretildiğini anlamasını engellemek (aradaki bitleri yutarak).
3. **Güvenlik:** İki basit ve güvensiz yapıdan (LFSR), tek bir karmaşık ve güvenli yapı (Shrinking Generator) elde etmek.

Yaptığınız iş aslında **kaos** üretmektir ve kriptografinin temeli de budur.

Kod ve İmplementasyon



1. Görüntüde Neyi İncelyoruz? (Görsel Analiz)

Ekranda iki yeşil çizgi (sinyal) görüyorsunuz: `stream_out` ve `valid_out`. Bunlar zamanla (soldan sağa) değişiyor.

`valid_out` Sinyali (Alttaki Çizgi)

- **Görevi:** Bu sinyal, sistemin o anki çıktısının "**geçerli**" olup olmadığını söyler.
- **Hikayesi:** Hatırlarsanız iki LFSR'miz vardı: **LFSR A (64-bit)** ve **LFSR B (63-bit)**.
 - LFSR A bizim "**Seçici**"mizdi.
 - Bu sinyal, LFSR A'nın ürettiği biti temsil eder.
- **Grafikteki Anlamı:** Çizgi yukarı çıktığında (1 olduğunda), LFSR A "Tamam, bu veriyi kullan" diyor. Çizgi aşağı indiğinde (0 olduğunda), LFSR A "Dur, bu veriyi çöpe at" diyor. Dikkat ederseniz bu çizgiler düzensiz aralıklarla inip çıkıyor; çünkü LFSR A rastgele (pseudo-random) bit üretiyor.

`stream_out` Sinyali (Üstteki Çizgi)

- **Görevi:** Bu, bizim **şifreli anahtar akışımız (keystream)**. Kriptografide kullanacağımız asıl veri budur.
- **Hikayesi:** Bu sinyal, **LFSR B**'den gelen veridir.
- **Grafikteki Anlamı:**
 - Dikkatli bakın: `valid_out` sinyali **1 (Yüksek)** olduğu anlarda, `stream_out` değişebilir (LFSR B'nin o anki değerini alır).
 - `valid_out` sinyali **0 (Düşük)** olduğunda, `stream_out` sinyali **değişmez**, son hali neyse öyle kalır (buna "latch" denir). Çünkü o sırada sistem "duraksıyor" veya veriyi "daraltıyor" (shrinking).

2. Kodlarda Ne Oldu? (Derinlemesine Analiz)

Şimdi yazdığınız VHDL kodunun bu grafiği nasıl çizdiğini, sanki bir makinenin içini açmışız gibi inceleyelim.

A. `lfsr.vhdl` (Motor Kısmı)

Bu dosya, tek başına bir sayı üreticiydi. Kodun kalbi şurasıydı:

```
-- Burası "XOR" işleminin yapıldığı yer.
-- Polinomda belirlediğimiz bitleri alıp karıştırıyor.
v_xor := v_xor xor r_reg(i);

-- Burası da kaydırma (Shift) işlemi.
-- Her saat vuruşunda (rising_edge) bitler bir sola kayıyor.
r_reg <= r_reg(WIDTH-2 downto 0) & feedback;
```

Anlamı: Bu kod her saat vuruşunda (Clock) register içindeki bitleri karıştırıp bir sola öteliyor. Tıpkı bir piyango küresinin her turda dönüp yeni bir top çıkarması gibi. Siz iki tane (64 ve 63 bitlik) küre oluşturdunuz.

B. `shrinking_gen.vhdl` (Beyin Kısmı - Karar Mekanizması)

Asıl büyü burada gerçekleşiyor. Grafikteki o düzensizliği yaratan kod bloğu şuydu:

```
process(clk, rst)
begin
    -- ... reset kontrolleri ...
    elsif rising_edge(clk) then
        -- KRİTİK NOKTA BURASI
        if bit_A_select = '1' then
            stream_out <= bit_B_data;
            valid_out <= '1';
        else
            valid_out <= '0';
        end if;
    end if;
end process;
```

Grafikle Eşleştirme: Simülasyon görüntüsünde `valid_out` 'un 0 olduğu (çukurda olduğu) zaman dilimlerine bakın. O anlarda `stream_out` çizgisinin hiç değişmediğini, bir önceki durumu koruduğunu göreceksiniz. İşte bu, yukarıdaki `else` bloğunun çalışmasından kaynaklanıyor. Sistem o an "kör" oluyor ve veri üretmiyor.

3. Neden Bunu Yaptık? (Mühendislik ve Kripto Açısı)

Birisi size "Neden tek bir LFSR kullanmadık da bu kadar uğraştık?" diye sorarsa cevabınız şu olmalı:

1. Doğrusallığı Bozmak (Non-Linearity):

- Tek bir LFSR tamamen matematiktir (Doğrusal). Eğer ben sizin ürettiğiniz ardışık $2N$ biti yakalarsam (örneğin 128 bit), Berlekamp-Massey algoritması denen bir yöntemle şifrenizi **saniyeler içinde** çözerim.
- Ancak **Shrinking Generator**'da aralardan rastgele bitler sildik (`valid_out = 0` olduğu anlar).
- Bir saldırgan (hacker) dışarıdan bakınca, hangi bitin LFSR B'den geldiğini, hangi bitin arada silindiğini bilemez. Aradaki zaman algısını yok ettiniz.

2. Uzun Periyot:

- 64 ve 63 sayıları aralarında asal olduğu için, bu sistemin kendini tekrar etmesi (periyodu) $2^{64} \times 2^{63} \approx 2^{127}$ saat vuruşu sürer. Bu, evrenin yaşından daha uzun bir süredir.

Nasıl Çalışıyor

- brew install ghdl

Proje Tasarımı: Shrinking Generator

Fikriniz kriptografik açıdan çok mantıklı.

- Neden 64 ve 63 bit?** 64 (2^6) ve 63 ($3^2 \times 7$) aralarında asaldır (co-prime). Bu, iki LFSR'nin ürettiği dizilerin çakışma periyodunun maksimum olmasını sağlar (yaklaşık 2^{127} bitlik bir periyot).
- Shrinking Generator Mantığı:**
 - LFSR A (Seçici - 64 bit):** Bir bit üretir (S).

- **LFSR B (Üreteç - 63 bit):** Bir bit üretir (G).
- **Kural:** Eğer S = '1' ise, G'yi çıktı olarak al (Z = G). Eğer S = '0' ise, G'yi çöpe at ve çıktı verme (hiçbir şey üretme).

3. Kullanmanız Gereken Polinomlar (Tap Points)

Araştırmanızı hızlandırmak için Xilinx Application Note (XAPP 052) standartlarına uygun **Maksimal Uzunluklu (Primitive)** polinomları sizin için listeledim. Kodunuzda bu "tap" noktalarını kullanmalısınız.

1. LFSR 64-Bit (Seçici):

- **Polinom:** $x^{64} + x^4 + x^3 + x + 1$
- **VHDL Tap'leri:** 64, 4, 3, 1. (VHDL array indekslemesinde genellikle 63 downto 0 kullanılır, buna göre ayarlama yapmalısınız: `bit(63) XOR bit(3) XOR bit(2) XOR bit(0)` gibi).

2. LFSR 63-Bit (Üreteç):

- **Polinom:** $x^{63} + x^1 + 1$
- **VHDL Tap'leri:** 63, 1.

4. Yol Haritası ve Kod Yapısı

VHDL kodlamaya başladığınızda şu hiyerarşiyi kurmanızı öneririm:

1. Dosya 1: `lfsr_generic.vhdl`

- Bu modülü "Generic" yapıda yazın. Yani bit genişliğini ve Tap noktalarını dışarıdan parametre olarak alabilsin. Böylece hem 64 hem 63 için aynı kodu kullanırsınız.

2. Dosya 2: `shrinking_gen.vhdl` (Top Module)

- İçinde iki tane `lfsr_generic` çağırır (biri 64, biri 63 bit olarak map edilir).
- Basit bir `process` bloğu ile Shrinking mantığını (if Select='1' then...) uygular.

3. Dosya 3: `testbench.vhdl`

- Clock ve Reset sinyallerini üretip sistemi koşturur.

Simülasyon Adımları (Terminalde Çalıştırılacak)

VS Code terminalini açın (Terminal -> New Terminal), projenin olduğu klasörde olduğunuzdan emin olun ve sırasıyla şu komutları girin:

1. Kodları Analiz Et (Derle):

Bash

```
ghdl -a lfsr.vhdl shrinking_gen.vhdl tb_shrinking.vhdl
```

2. Modeli Oluştur (Elaborate):

Bash

```
ghdl -e tb_shrinking
```

3. Simülasyonu Çalıştır ve Dalga Dosyası (.vcd) Oluştur:

Bash

```
ghdl -r tb_shrinking --vcd=wave.vcd --stop-time=5us
```

VS Code İçinde "WaveTrace" Eklentisi (Önerilen)

Programdan çıkmadan dalga formlarını doğrudan VS Code içinde görebilirsiniz.

1. VS Code'da sol taraftaki **Extensions** (Kareler simgesi) menüsüne tıklayın.
2. Arama çubuğuna **WaveTrace** yazın.
3. Çıkan eklentiye (yayıncı: *wavetrace*) **Install** diyerek kurun.
4. Kurulum bitince, sol taraftaki dosya gezgininden oluşturduğumuz `wave.vcd` dosyasına **çift tıklayın**.
5. Dalga formları VS Code içinde yeni bir sekmede açılacaktır.

Nasıl Kullanılır:

- Sinyallerin üzerine gelerek değerleri okuyabilirsiniz.
- `stream_out` ve `valid_out` sinyallerini inceleyin. `valid_out` '1' olduğunda `stream_out` 'un nasıl değiştiğini gözlemleyin.

Hocaya yazılacak

Selamlar Hocam.

Tavsiyeniz üzerine LFSR ve Shrinking Generator implementasyonu ile ilgili çalışmalarını tamamladım. Geldiğim noktada:

Bağımsız LFSR Tasarımı: İsteğiniz üzerine 64-bit ve 63-bit uzunluğunda iki bağımsız LFSR modülü tasarladım. Bu uzunlukları, birbirleriyle aralarında asal (coprime) oldukları için seçmiştik.

Polinom Seçimi: Geri besleme için endüstri standardı olan İlkel Polinomları (Primitive Polynomials) kullandım. Bu sayede her iki LFSR da kendi maksimum döngü uzunluğuna ulaşabiliyor.

Shrinking Generator Yapısı: Tasarladığım bu iki LFSR'yi, Shrinking Generator mimarisinde birleştirdim. 64-bitlik LFSR'yi "Seçici (Selector)", 63-bitlik LFSR'yi ise "Veri (Data)" kaynağı olarak kullandım. Seçiciden gelen bit '1' ise veriyi çıkışa aktaran, '0' ise veriyi yutan (discard) asenkron yapıyı VHDL ile modelledim.

Simülasyon ve Test: Tasarımı GHDL ortamında derleyip simüle ettim. Elde ettiğim dalga formlarında (waveform), valid sinyalinin düzensiz aralıklarla üretildiğini ve sistemin beklenen doğrusal olmayan (non-linear) davranışı sergilediğini doğruladım.

Şimdi neler yapmamı istersiniz hocam.